hashlock.

2026

Security Audit

Celo 4th (DeFi)

March 2026

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	17
Conclusion	18
Our Methodology	19
Disclaimers	21
About Hashlock	22

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

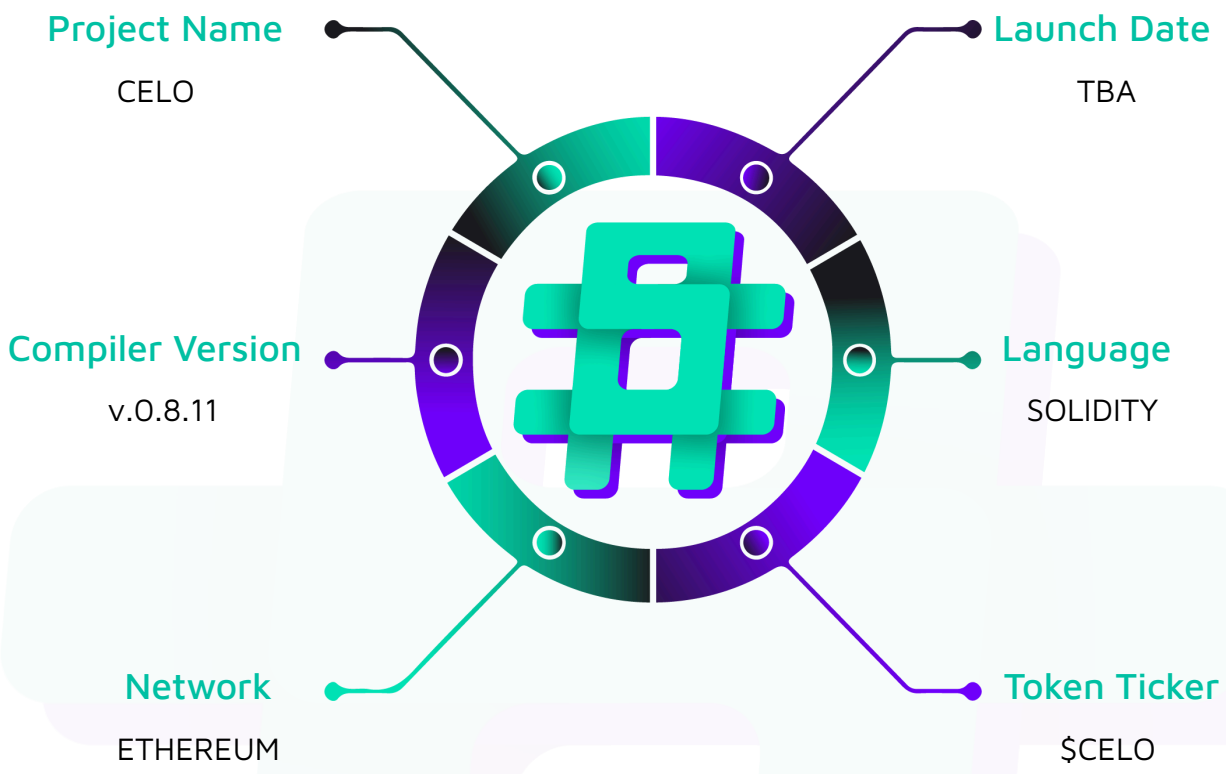
The Celo team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

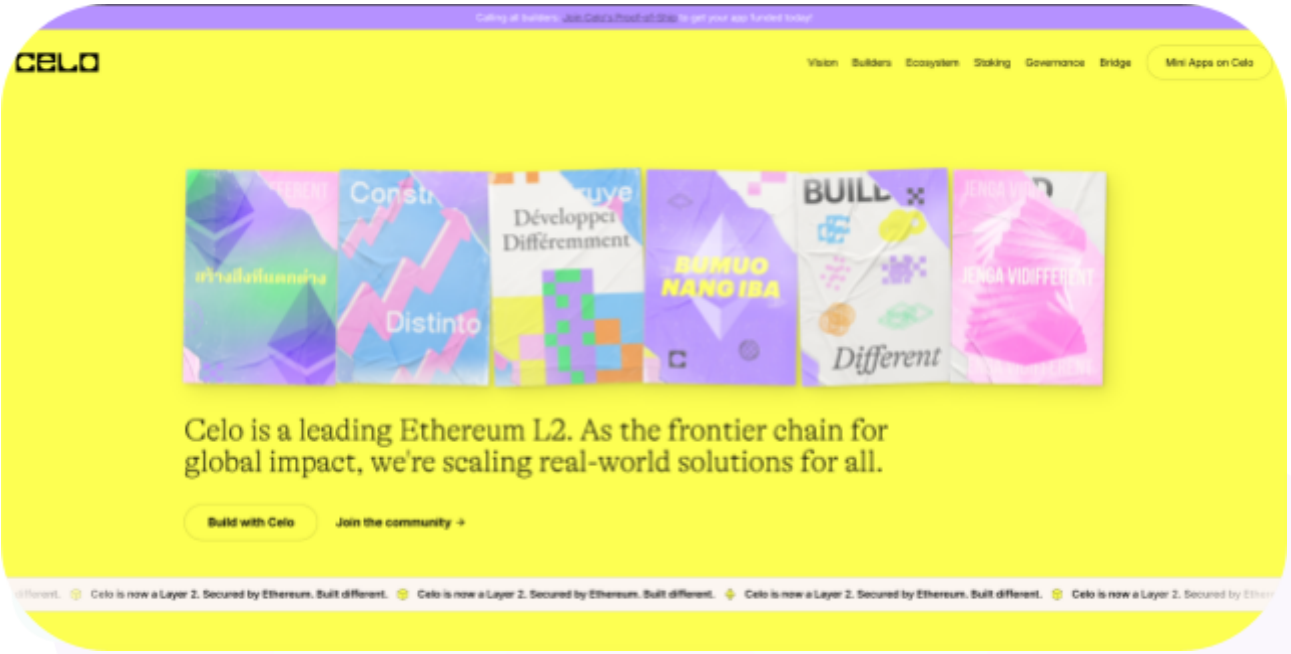
The Celo project is a purpose-driven, Ethereum-anchored Layer-2 blockchain platform designed to enable fast, low-cost, and carbon-aware payments and decentralized finance (DeFi) applications globally. It supports gas payments in ERC-20 tokens and incorporates modular technologies, including an OP-Stack L2, EigenDA for scalable data availability, and a zkEVM (via Succinct SP1) for verified execution, together enabling one-second block times and sub-cent fees. By prioritizing accessibility, interoperability with Ethereum, and sustainability (for example, allocating 20 % of transaction fees to carbon offsets), Celo positions itself as an infrastructure foundation for real-world financial inclusion and on-chain economic growth.

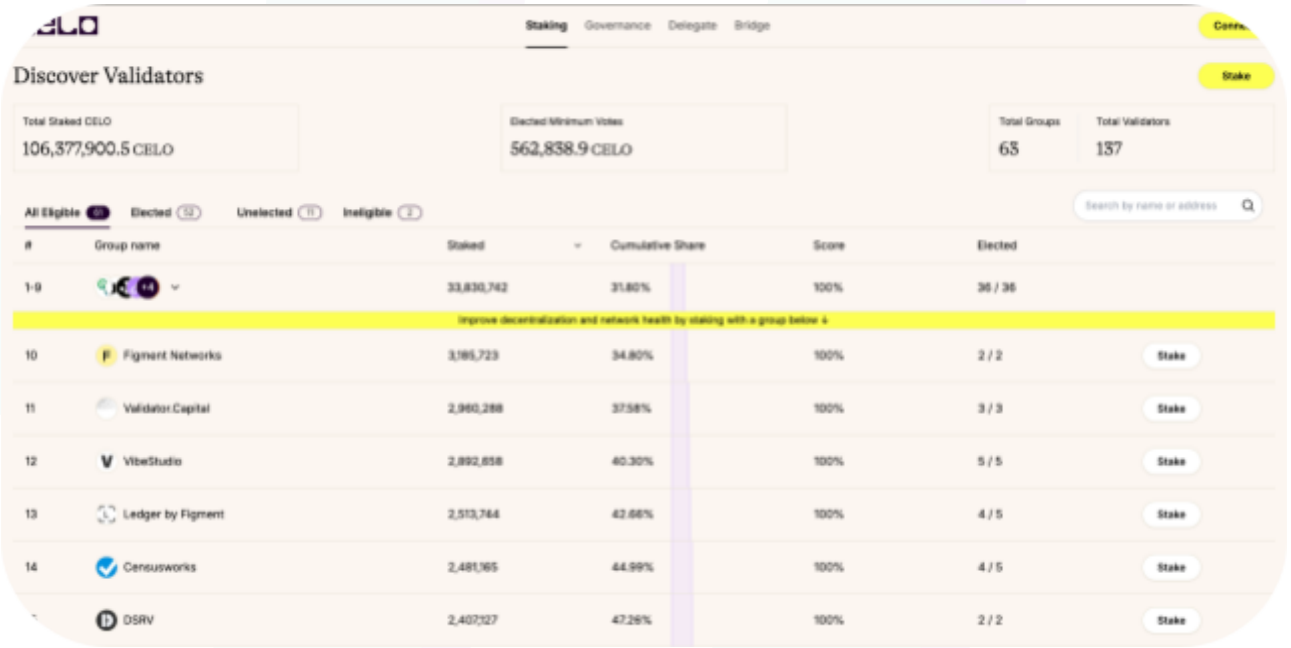
Project Name: Celo
Project Type: DeFi
Compiler Version: 0.8.11
Website: https://celo.org/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the Celo project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Celo Smart Contracts**
Network	**Ethereum**
Language	**Solidity**
Audit Date	**March, 2026**
Contract 1	CeloSuperchainConfig.sol
Audited GitHub Commit Hash	39e96786cefc75bec597b229f339262fc6ec72f6
Fix Review GitHub Commit Hash	f574233e574df6122459ac589fe7141b4c8c637d

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

1 Gas Optimisation

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
ICeloSuperchainConfig.sol Allows users to: - Query pause state (blanket and per-identifier) - Query guardian address and SuperchainConfig address - Check and propagate pause from upstream SuperchainConfig Allows admins to: - Initialize with guardian, pause flag, and SuperchainConfig address - Pause the system with an identifier string - Unpause the system	**Contract achieves this functionality.**
CeloSuperchainConfig.sol Allows users to: - Query blanket pause status via `paused()` (two-tier: local + upstream SuperchainConfig) - Query per-identifier pause status via `paused(address)` (local blanket OR upstream per-identifier) - Query local-only pause status via `celoPaused()` - Query guardian address and SuperchainConfig address - Trigger upstream pause propagation via `checkAndPauseIfSuperchainPaused()` Allows admins to: - Initialize contract with guardian, paused flag, and SuperchainConfig address (once, via proxy) - Pause withdrawals via `pause()` (guardian only)	**Contract achieves this functionality.**

- Unpause withdrawals via `unpause()` (guardian only) - Change guardian or SuperchainConfig address only via contract upgrade	

Code Quality

This audit scope involves the smart contracts of the Celo project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Celo project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

QA

[Q-01] CeloSuperchainConfig#paused - Insufficient NatSpec on Forward-Compatibility Getter

Description

The newly added `paused(address _identifier)` function has a single-line NatSpec (`/// @notice Forward-compatibility getter`) with no `@param`, `@dev`, or `@return` tags. The function implements a two-tier pause hierarchy — Celo's local blanket pause checked first via `celoPaused()`, then upstream SuperchainConfig's per-identifier state — but this cascading behavior is not documented. Other functions in the same contract, such as the no-arg `paused()`, have multi-line documentation describing their behavioral nuances.

Recommendation

Add NatSpec describing the two-tier pause hierarchy and parameter semantics:

```
/// @notice Forward-compatibility getter for per-identifier pause state.
/// @dev Implements a two-tier pause check: (1) Celo local blanket pause via
///      celoPaused(), (2) upstream SuperchainConfig per-identifier state.
///      When celoPaused() is true, returns true for all identifiers.
/// @param _identifier The address identifier forwarded to
///        ISuperchainConfig.paused(address) when the local system is not paused.
function paused(address _identifier) public view returns (bool paused_) {
```

Status

Resolved

Gas

[G-01] CeloSuperchainConfig#paused - Redundant Storage Read Due to Uncached `superchainConfig()` Call

Description

In `paused(address _identifier)`, `superchainConfig()` is called twice within the same execution — once for the zero-address guard and once for the actual `ISuperchainConfig.paused(_identifier)` delegation. Each invocation reads `SUPERCHAIN_CONFIG_SLOT` via `Storage.getAddress()`, resulting in an unnecessary warm SLOAD (~100 gas). The same pattern exists in the no-arg `paused()` function, while `checkAndPauseIfSuperchainPaused()` already correctly caches the result in a local variable.

Recommendation

Cache `superchainConfig()` in a local variable, consistent with the pattern used in `checkAndPauseIfSuperchainPaused()`:

```
 function paused(address _identifier) public view returns (bool paused_) {
      paused_ = celoPaused();
      if (paused_) {
          return paused_;
      }

-     if (superchainConfig() != address(0)) {
-         paused_ = ISuperchainConfig(superchainConfig()).paused(_identifier);
+     address sc = superchainConfig();
+     if (sc != address(0)) {
+         paused_ = ISuperchainConfig(sc).paused(_identifier);
      }

      return paused_;
  }
```

Status

Resolved

Centralisation

The Celo project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Celo project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au

hashlock.